

Final Term

1. Sorting Algorithm

Insertion Sort:

1. Define Insertion Sort.

Ans: Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.

2. Algorithm Step:

Step 1: We start with second element of the array as first element in the array is assumed to be sorted.

Step 2: Compare second element with the first element and check if the second element is smaller then swap them.

Step 3: Move to the third element and compare it with the first two elements and put at its correct position.

Step 4: Repeat until the entire array is sorted.

3. Step by Step simulation for array.

Selection Sort:

4. Define Selection Sort Algorithm.

Ans: Selection Sort is a comparison-based sorting algorithm. It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element.

Time Complexity: $O(n^2)$

5. Algorithm Step:

Step 1: First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.

Step 2: Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.

Step 3: We keep doing this until we get all elements moved to correct position.

6. Step by Step simulation for array.

Counting Sort:

8. Define Counting Sort.

Ans: Counting Sort is a non-comparison-based sorting algorithm. It is particularly efficient when the range of input values is small compared to the number of elements to be sorted.

9. Algorithm Step:

Step 1 - Maintain two arrays, one with the size of input elements without repetition to store the count values and other with the size of the input array to store the output.

Step 2 - Initialize the count array with all zeroes and keep the output array empty.

Step 3 - Every time an element occurs in the input list, increment the corresponding counter value by 1, until it reaches the end of the input list.

Step 4 - Now, in the output array, every time a counter is greater than 0, add the element at its respective index.

Step 5 - Repeat Step 4 until all the counter values become 0. The list obtained is the output list.

10. Step by Step simulation for array.

2. Greedy Algorithm

12. Define Greedy Algorithm.

Ans: Greedy algorithms build up a solution step by step, always choosing the option that offers the most immediate benefit. Used for optimization problems where local choices lead to a globally optimal solution.

If both of the properties below are true, a greedy algorithm can be used to solve the problem.

Greedy choice property: A global (overall) optimal solution can be reached by choosing the optimal choice at each step.

Optimal substructure: A problem has an optimal substructure if an optimal solution to the entire problem contains the optimal solutions to the sub-p

13. Characteristics of Greedy Algorithm.

1. Greedy choice property
2. Optimal substructure
3. No backtracking
4. Local optimization
5. Making decisions based on the best available option at each step.

14. Advantages of Greedy Algorithms:

1. Simple and easy to understand
2. Efficient for certain problems
3. Fast execution
4. Can be used as building blocks for more complex algorithms

Disadvantages of Greedy Algorithms:

1. Does not work for all optimization problems.
2. Can lead to suboptimal solutions in some cases.
3. Requires careful analysis before implementation.

15. Define Fractional Knapsack.

Ans: Given the weights and profits of N items, put these items in a knapsack of capacity W to get the maximum total profit in the knapsack.

In Fractional Knapsack, we can break items for maximizing the total value of the knapsack.

3. Huffman Coding

19. Huffman Coding Application:

1. Huffman coding is used in traditional compression techniques like GZIP etc.
2. For text and fax transmissions

4. String Matching Algorithm

20. Define String Matching Algorithm.

Ans: String Matching Algorithms also known as String Searching Algorithms form an essential category of string algorithms. They are defined as methods used to identify the position where one or more specific strings occur within a larger string.

21. The Naïve String-Matching Algorithm.

Ans: The Naive String-Matching Algorithm is a straightforward and simple method for finding occurrences of a pattern within a larger text.

Steps of the Algorithm:

1. Start at the beginning of the text.
2. Compare the pattern with a substring of the text, character by character.
3. If all characters in the pattern match the corresponding characters in the text, a match is found.
4. Move the pattern one position to the right in the text and repeat the comparison until the end of the text is reached.
5. Return the positions where matches are found.

Naive String-Matching Example.

22. Define Rabin-Karp Algorithm.

Ans: The Rabin-Karp Algorithm is an efficient string-matching algorithm that uses hashing to find a pattern within a text.

Steps of Rabin-Karp Algorithm:

1. Compute the hash value of the pattern using a specific hash function.
2. Compute the hash value for the first substring of the text with the same length as the pattern.
3. Compare the hash of the pattern with the hash of the current substring of the text. If the hashes match, perform a character-by-character comparison to confirm the match (to avoid false positives caused by hash collisions).
4. Slide the pattern over the text one position at a time, updating the hash and checking for matches

Formula and Hash Calculation Example.

The formula is:

$$\text{Hash (S)} = (c_0 \cdot p^{m-1} + c_1 \cdot p^{m-2} + \dots + c_{m-1} \cdot p^0) \bmod q$$

Where:

- S is the string (e.g. the pattern or a substring of the text).
- $c_0, c_1, \dots, c_{m-1}$ are the numeric values of the characters in the string S.
For example, if the string contains ASCII characters, these are their ASCII values.
- p is a prime number or a base value used in the polynomial (e.g. 31 or 256).
- m is the length of the string S.
- q is a large prime number used to reduce hash collisions by taking the modulus.

5. Recursion

23. Assignment -> 1 to 6 questions.

24. i. Define Tail Recursion.

Ans: The recursive call is the last operation before returning the final value.

```
void tailFactorial(int n, int result){
    if (n == 0) {
        cout << "Factorial: " << result << endl;
        return;
    }
    tailFactorial(n - 1, result * n); // Recursive call is the last operation
}
```

ii. Define Head Recursion

Ans: The recursive call is the first operation before any other computation.

```
void headRecursion(int n){
    if (n == 0)
        return;
    headRecursion(n - 1);    // Recursive call happens first
    cout << n << endl;      // Processing after recursion
}
```

25. Recursion code – i. Fibonacci, ii. Factorial code

26. Recursion tree for Fibonacci.

6. Backtracking

27. Define Backtracking Algorithm with example

Ans: A backtracking algorithm works by recursively exploring all possible solutions to a problem. It starts by choosing an initial solution, and then it explores all possible extensions of that solution. If an extension leads to a solution, the algorithm returns that solution.

Example: N-Queens Problem, Graph Coloring, puzzles and path finding.

28. Algorithm step:

Step -1: Choose an initial solution.

Step -2: Explore all possible extensions of the current solution.

Step -3: If an extension leads to a solution, return that solution.

Step -4: If an extension does not lead to a solution, backtrack to the previous solution and try a different extension.

Step -5: Repeat steps 2-4 until all possible solutions have been explored.

29. N queen Problem explain (4/5/6).

Advantage of Backtracking Algorithm.

1. Backtracking can be used to solve complex combinatorial optimization problems. It can be adapted to various constraints and problem-specific requirements.
2. Backtracking can be optimized through pruning.
3. Backtracking algorithms can be parallelized and executed in multiple threads.

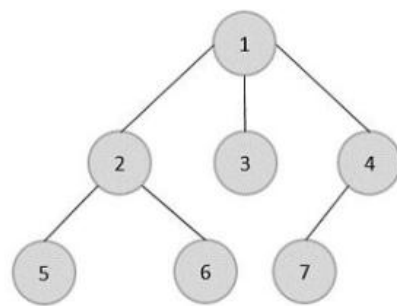
Disadvantage of Backtracking Algorithm.

1. Backtracking can be very slow for large problems.
2. Backtracking can also be memory-intensive, as it requires storing the state of the partial solution at each step of the search.
3. Backtracking can be inefficient, as it may explore the same parts of the search space multiple times.

7. Tree Basic

30. Define tree Data structure with example.

Ans: A tree is a non-linear abstract data type with a hierarchy-based structure. It consists of nodes that are connected via links.



General Tree Data Structure

31. Why use a Tree Data Structure?

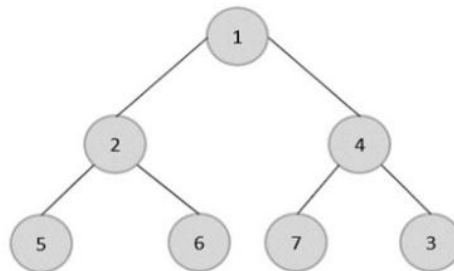
Ans: Linear data structures like arrays, linked lists, stacks, and queues store data sequentially. As the data size grows, the time complexity for performing operations also increases, which is often inefficient in modern computational scenarios.

32. Assignment -> 7 to 10

8. Type of Tree

33. Define Binary tree with example.

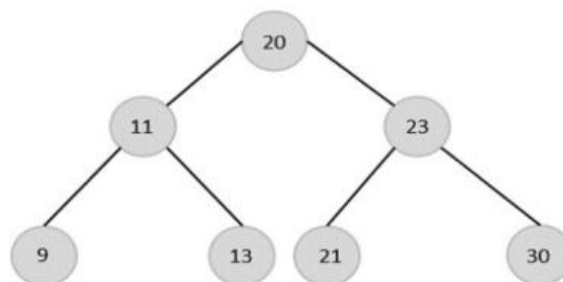
Ans: A binary tree is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. Binary trees are a foundational structure in computer science and are widely used in various applications.



Binary Tree Data Structure

34. Define Binary Search Tree with example.

Ans: The data in the Binary Search Trees (BST) is always stored in such a way that the values in the left subtree are always less than the values in the root node and the values in the right subtree are always greater than the values in the root node, i.e. $\text{left subtree} < \text{root node} \leq \text{right subtree}$.



Binary Search Tree Data Structure

35. Assignment question: 11 to 13

9. Shortest Path Algorithms

36. Dijkstra Algorithm Steps.

1. Mark the source node with a current distance of 0 ($d[s]=0$) and assign infinity (∞) to all other nodes. 2. Set the unvisited node with the smallest current distance as the current node (u).

3. For each neighbor v of u:

If ($d[u]+c[u,v]<d[v]$)

$d[v]=d[u]+c[u,v]$

Here:

$c[u,v]$: Cost (weight) of the edge from u to v. $d[v]$: Distance to v.

$d[u]$: Distance to u

4. Mark the current node u as visited.

5. Go back to Step 2 if there are any unvisited nodes.

37. Dijkstra's algorithm for finding the shortest path from source to destination.